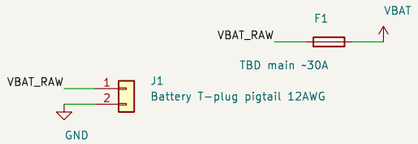
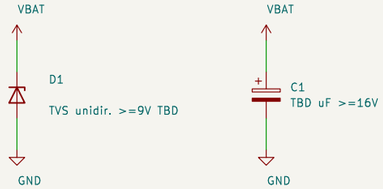
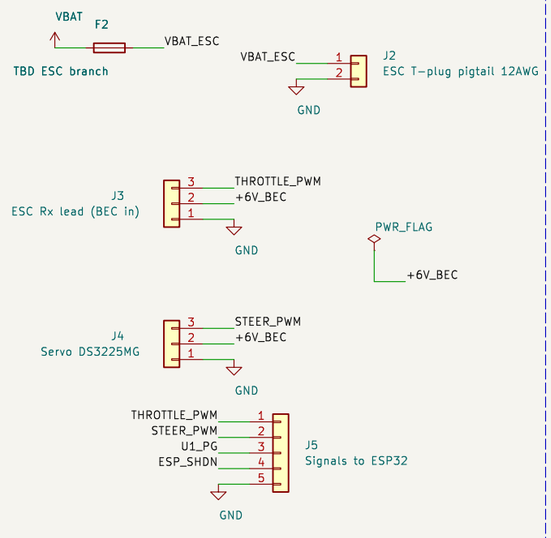
A Battery input (2S LiPo, 6.0–8.4 V)



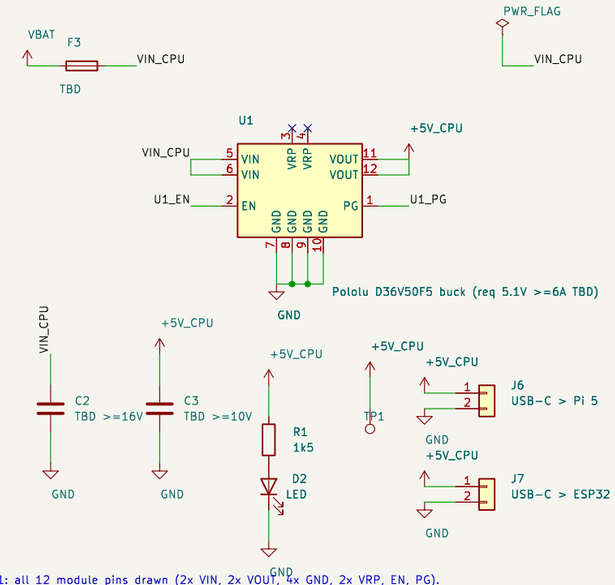


B ESC and servo (servo power = ESC BEC, 6 V / 3 A)



+6V_BEC comes only from the ESC BEC; never join a 5 V rail.
J5: THROTTLE_PWM = GPIO5, STEER_PWM = GPIO4,
U1_PG and ESP_SHDN = GPIO TBD (3.3 V logic, see region F).

C Compute 5 V (+5V_CPU: Pi 5 + ESP32 only)



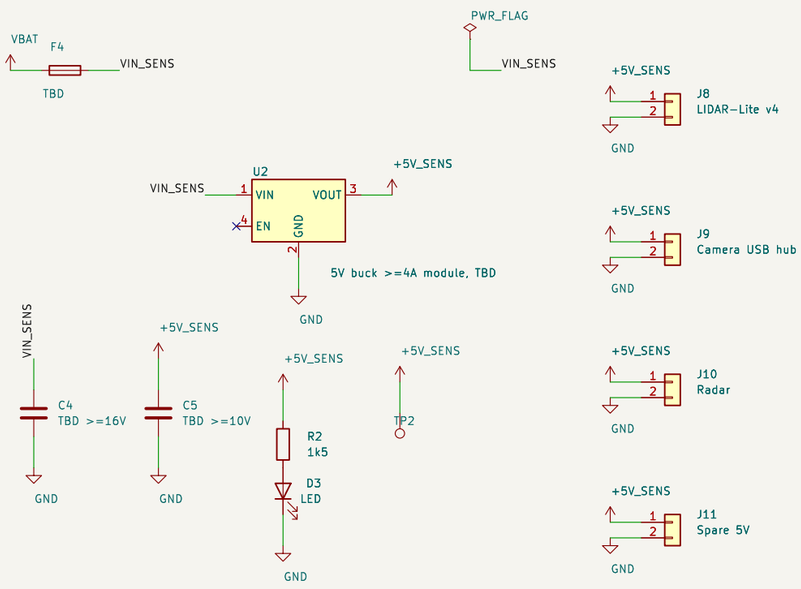
U1: all 12 module pins drawn (2x VIN, 2x VOUT, 4x GND, 2x VRP, EN, PG).
VRP not used: sensor buck is fed from VBAT via F4 (decided).
J6/J7: 2-wire USB-C pigtails into the Pi 5 and ESP32 USB-C inputs,
so each board's own USB-C input protection is used.
EN/PG control: region F.

Notes

1. Battery: 2S LiPo, 6.0–8.4 V (ESC LVC cuts at 6.0 V). Battery and ESC use T-plug pigtails (12 AWG bare ends) soldered to J1/J2 pads; pad size TBD (12 AWG is 3.3 mm2, footprint is the 2.5 mm2 placeholder).
2. All fuses and protection values TBD; positions are proposals.
3. Single common GND; star point at J1 in layout.
4. +6V_BEC is ESC-supplied, isolated from 5 V rails.
5. Signals are 3.3 V logic or lower.
6. Rails are project power symbols (rc_power lib): pin name = Value = net name (KiCad 7–10 safe).
7. PWR_FLAG on VBAT_RAW, GND, VBAT, VIN_CPU, VIN_SENS, LDO_IN, +6V_BEC.
8. Net classes (kicad_pro): HighCurrent = *VBAT*, GND; Power5A = +5V_CPU, *VIN_CPU; Power3A = +5V_SENS, *VIN_SENS, *+6V_BEC. 20 A path: zones or off-board wire (open Q1).
9. J6/J7 USB-C pigtails are 24 AWG: about 0.25 V drop at 5 A over 30 cm, keep them short. The Pi 5 gets no USB-PD 5 A offer this way (USB ports limited to 600 mA unless set in config.txt).
10. Spec: docs/schematics/power_board_plan.md. Values marked TBD are unconfirmed.

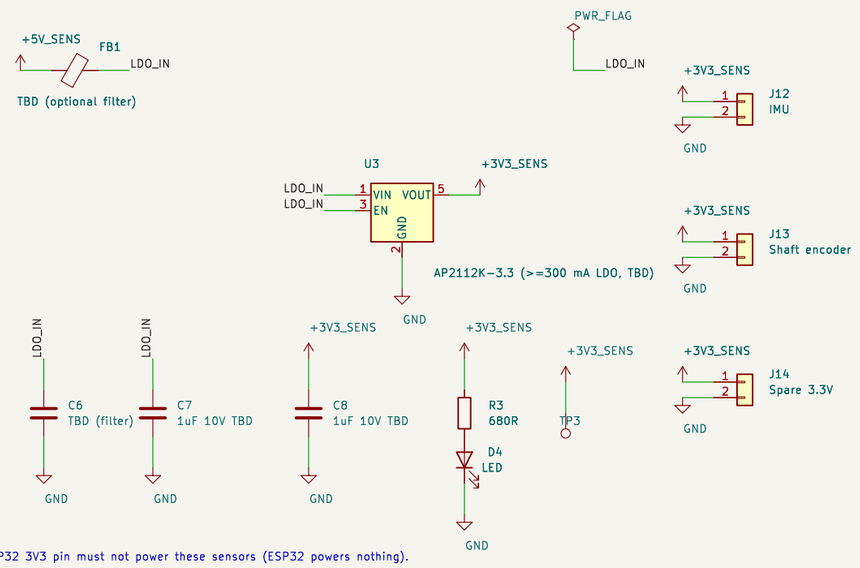


D Sensor 5 V (+5V_SENS: heavy sensors + 3.3 V LDO)



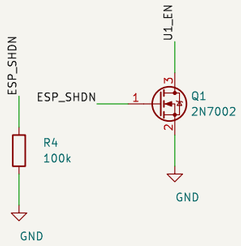
U2 EN: per module datasheet, TBD. U2 footprint TBD (module pinout).

E Sensor 3.3 V (+3V3_SENS: LDO from +5V_SENS)



ESP32 3V3 pin must not power these sensors (ESP32 powers nothing).

F U1 EN / PG to ESP32 (proposal, open Q13)



U1 EN (D36V50F5): off below 1.2 V, on above 1.35 V.
EN has a 100k pull-up to VIN inside the module (up to 8.4 V), so EN must NEVER go straight to an ESP32 GPIO (8.4 V on a 3.3 V pin while the ESP32 is off or booting). U1 also powers the ESP32, so U1 must start on its own: it stays ON by default.
Proposed: ESP_SHDN high -> Q1 pulls EN low -> U1 off. This is a POWER CYCLE, not a shutdown: the ESP32 loses power, R4 turns Q1 off, U1 restarts and the Pi 5 and ESP32 reboot (hard cut to the Pi). A true low-battery cutoff needs a latch or a VIN divider on EN (open Q13).
R4 holds Q1 off while the ESP32 boots or is absent.
ESP_SHDN: non-strapping GPIO, no pull-up at reset.
U1_PG: non-strapping GPIO (PG is low while VOUT ramps).
U1_PG: open-drain, low when VOUT is >20% high or >10% low. Pull-up: ESP32 INPUT_PULLUP (3.3 V).
GPIOs for U1_PG / ESP_SHDN: TBD (pins.h + README).

Baseline generated programmatically; project KiCad version 10.0
Spec: docs/schematics/power_board_plan.md
Draft for review, values marked TBD are unconfirmed
gvo27/racecar

Sheet: /		
File: rc_power_board.kicad_sch		
Title: RC Car Power Distribution Board		
Size: A3	Date: 2026-10-28	**Rev: A (draft)**
KiCad E.D.A. kicad-cli 7.0.11+dfsg-1build4		Id: 1/1